To preserve a user's login for 12 days, you must issue a persistent, secure **token-based authentication cookie** or use **JSON Web Tokens (JWT)** stored in the browser's `localStorage`. You will set the cookie's expiration to exactly 12 days so the browser retains it even if the user closes the window or restarts their computer. Stack Overflow +3

Here is how you can implement this functionally on your site:

1. Set the Persistent Token

When the user successfully logs in and checks the **"Remember Me"** option, generate a secure, randomized token on your backend and store it in your database alongside the user's ID. Then, send this token to the client in an HTTP-only cookie. PHPpot +1

In JavaScript (Vanilla JS):

javascript

```javascript
function setLoginCookie(token, days = 12) {
    const date = new Date();
    // Calculate 12 days in milliseconds
    date.setTime(date.getTime() + (days * 24 * 60 * 60 * 1000));
    const expires = "expires=" + date.toUTCString();

    // Set the cookie
    document.cookie = "auth_token=" + token + ";" + expires + ";path=/;secure;sar
}
```

Use code with caution.

In Node.js/Express (Backend):

javascript

```javascript
res.cookie('auth_token', token, {
    maxAge: 12 * 24 * 60 * 60 * 1000, // 12 days in milliseconds
    httpOnly: true,  // Prevents JavaScript from accessing the cookie
    secure: true,    // Requires HTTPS
    sameSite: 'Strict'
});
```

Use code with caution.

Note: Using `httpOnly: true` in your backend is a crucial Stack Overflow best practice to prevent Cross-Site Scripting (XSS) attacks.

2. Handle the Re-Login on Page Load

On every page request, your backend (or frontend route guard) should check for the presence of this cookie. PHPpot +3

1. If the cookie exists, verify the token against your database.
2. If it is valid, silently log the user in and refresh the 12-day expiration timer.
3. If the token is not found or expired, force them to the login screen.

3. Clear the Cookie on Logout

When a user manually clicks **Logout**, you must delete the cookie by setting its expiration date to a time in the past and invalidating the token from your backend database. YouTube · Bill Moseley, Ph.D. - Compute… +1

javascript

```javascript
function logoutUser() {
    // Overwrite the cookie with an expired date
    document.cookie = "auth_token=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/
    // Redirect to login page
    window.location.href = "/login";
}
```

Use code with caution.

4. Security Considerations

Because 12 days is a significant amount of time to stay logged in, consider these implementations:

- **Token Rotation:** Issue a new token every time the user authenticates, making stolen tokens harder to use. User Experience Stack Exchange +2

- **Re-authentication for Sensitive Actions:** Force users to re-enter their passwords before changing account details, viewing payment info, or modifying passwords. Webmasters Stack Exchange +3
- **Browser Session Storage:** For more on implementing persistent logins efficiently, refer to the Medium or OutSystems implementation guides.

If you want to move forward with the implementation, please let me know:

- What **backend technology or framework** is your website built with (e.g., Node.js, PHP, Python, React)?
- Do you already have a **database table** designed for handling user sessions?
- Are you using **token-based authentication (JWT)** or traditional server-side sessions?